CloudSim Web — Full Project Documentation

> *A browser-based interface built on top of CloudSim 7.0.1, the Java cloud computing simulation framework. Users configure and run cloud infrastructure simulations through a React UI instead of editing Java source files.*

Table of Contents

1. What This Project Is
2. High-Level Architecture
3. Project Layout
4. Backend — Spring Boot REST API
   - 4.1 Maven Configuration
   - 4.2 Application Properties
   - 4.3 Authentication — JWT Cookie Flow
   - 4.4 Security Filter Chain
   - 4.5 Simulation Lifecycle
   - 4.6 SimulationMapper — Java Object Construction
   - 4.7 SimulationRunner — Executing CloudSim
   - 4.8 Parameter Sweep
   - 4.9 Templates
   - 4.10 Database Schema
   - 4.11 REST API Contract
5. Frontend — React TypeScript App
   - 5.1 Tech Stack
   - 5.2 Vite Dev Proxy
   - 5.3 API Client — lib/api.ts
   - 5.4 React Hooks Layer
   - 5.5 Page: Landing
   - 5.6 Page: Login / Register
   - 5.7 Page: Dashboard
   - 5.8 Page: New Simulation
   - 5.9 Page: Simulation Result
   - 5.10 Page: Analytics
   - 5.11 Page: Compare
   - 5.12 Page: Parameter Sweep
   - 5.13 Page: Saved Configs
6. CloudSim Core Concepts
   - 6.1 Domain Objects
   - 6.2 Scheduling Policies
   - 6.3 Utilization Models
   - 6.4 Power Models
7. Configuration Parameter Reference
8. Advanced Features — Tier 2
   - 8.1 Power Modeling and Energy Tracking
   - 8.2 Failure Injection
   - 8.3 SLA Deadlines
9. Key Design Decisions

10. How to Run the Project
11. Pre-built Templates Reference
12. Data Flow — End to End

---

1. What This Project Is

CloudSim is a widely-used Java framework for modeling and simulating cloud computing environments. It lets researchers define virtual machines, cloudlets (tasks), datacenters, and hosts programmatically — then runs a discrete-event simulation to measure execution time, cost, throughput, and resource utilization.

The problem: Using CloudSim normally means writing and compiling Java code for every experiment. This project wraps CloudSim with a Spring Boot REST API and a React frontend so that anyone can:

- Configure a cloud environment through a form UI in a browser.
- Submit a simulation and watch it run asynchronously.
- View rich results: timing charts, Gantt timelines, cost breakdowns, energy graphs, and failure events.
- Compare two simulations side-by-side.
- Run parameter sweeps: vary one dimension (e.g. VM count) across a range and see how metrics change.
- Save and reload configurations.

The original CloudSim source code (`modules/cloudsim/`) is **completely untouched**. All new code lives in:

- `modules/cloudsim-web/` — the Spring Boot wrapper
- `web/` — the React frontend

---

2. High-Level Architecture

```
┌──────────────────────────────┐   HTTP / JSON
┌────────────────────────────────────────┐
│  React Frontend  (web/)       │ ←──────────────→ │  Spring Boot API  (cloudsim-web/)
│
│  Vite + TypeScript + Tailwind │                  │  Java 21 · Spring Boot 3.3.5
│
│  TanStack Query               │                  │  Spring Security (JWT httpOnly
cookie) │
│  React Router v6              │                  │  Spring Data JPA → SQLite
│
│  Recharts · shadcn/ui         │                  │  CloudSim 7.0.1 (Maven dependency)
│
│  react-hook-form · Zod        │                  │  Single-thread executor
(serialised)   │
└──────────────────────────────┘
└────────────────────────────────────────┘
         localhost:5173                                          localhost:8080
              │                                                        │
              └──────── /api/** proxied by Vite during dev ────────────┘
```

Key constraint handled by design: CloudSim uses static global state (`CloudSim.init()` , global event queues). Two simulations **cannot** run concurrently in the same JVM. The backend serialises all runs through

a `newSingleThreadExecutor` — only one CloudSim simulation runs at a time, others queue behind it.

---

3. Project Layout

```
cloudsim-master/
├── modules/
│   ├── cloudsim/                  ← UNTOUCHED — CloudSim 7.0.1 core library
│   ├── cloudsim-examples/         ← UNTOUCHED — original Java examples
│   └── cloudsim-web/              ← NEW — Spring Boot REST API
│       ├── pom.xml
│       └── src/main/java/org/cloudbus/cloudsim/web/
│           ├── CloudSimWebApplication.java     ← entry point
│           ├── config/
│           │   ├── SecurityConfig.java         ← JWT + CORS + filter chain
│           │   └── JwtAuthFilter.java          ← reads cookie, sets SecurityContext
│           ├── auth/
│           │   ├── AuthController.java         ← /api/auth/* endpoints
│           │   ├── AuthService.java            ← register/login/logout logic
│           │   ├── JwtUtil.java                ← generate/validate/parse JWT
│           │   ├── RateLimitInterceptor.java   ← bucket4j rate limiting
│           │   └── RateLimitException.java
│           ├── model/
│           │   ├── SimulationRun.java          ← JPA entity (DB row)
│           │   ├── SimulationConfigDto.java    ← incoming JSON → Java
│           │   ├── DatacenterConfigDto.java
│           │   ├── HostConfigDto.java
│           │   ├── VmConfigDto.java
│           │   ├── CloudletConfigDto.java
│           │   ├── SimulationResultDto.java    ← outgoing result JSON
│           │   ├── SimulationSummaryDto.java
│           │   ├── SummaryStats.java
│           │   ├── CloudletResultDto.java
│           │   ├── VmStatsDto.java
│           │   ├── HostEnergyDto.java
│           │   └── FailureEventDto.java
│           ├── simulation/
│           │   ├── SimulationController.java   ← POST/GET /api/simulations
│           │   ├── SimulationService.java      ← submit, poll, build results
│           │   ├── SimulationRunner.java       ← runs CloudSim, captures output
│           │   ├── SimulationMapper.java       ← JSON DTOs → CloudSim objects
│           │   ├── SimulationRunRepository.java ← JPA CRUD
│           │   └── SimulationConfigValidator.java
│           ├── sweep/
│           │   ├── SweepController.java        ← POST /api/sweeps
│           │   ├── SweepService.java           ← loops runner over a range
│           │   ├── SweepConfigDto.java
│           │   └── SweepResultDto.java
│           └── templates/
│               ├── TemplateService.java        ← 9 hardcoded example configs
│               └── TemplateController.java     ← GET /api/templates
│
```

```
├── web/                            ← NEW — React + TypeScript frontend
│   ├── vite.config.ts              ← Vite build + proxy config
│   ├── tailwind.config.ts
│   ├── tsconfig.json
│   ├── package.json
│   └── src/
│       ├── main.tsx                ← React entry point
│       ├── App.tsx                 ← route definitions
│       ├── index.css               ← Tailwind directives + CSS vars
│       ├── lib/
│       │   ├── api.ts              ← typed fetch wrapper + all API calls + types
│       │   └── queryClient.ts      ← TanStack Query client
│       ├── hooks/
│       │   ├── useAuth.ts          ← login/logout/register/me
│       │   ├── useSimulations.ts   ← list/get/submit + template/config hooks
│       │   ├── useDarkMode.ts
│       │   └── useToast.ts
│       ├── components/
│       │   ├── RequireAuth.tsx     ← route guard
│       │   ├── ErrorBoundary.tsx
│       │   ├── Toaster.tsx
│       │   └── ui/                 ← shadcn/ui primitives (Button, Badge, Card, etc.)
│       └── pages/
│           ├── Landing.tsx
│           ├── Login.tsx
│           ├── Register.tsx
│           ├── Dashboard.tsx
│           ├── NewSimulation.tsx
│           ├── SimulationResult.tsx
│           ├── Analytics.tsx
│           ├── Compare.tsx
│           ├── ParameterSweep.tsx
│           └── SavedConfigs.tsx
│
├── docs/DESIGN.md                  ← original design document
├── PROJECT_DOCUMENTATION.md        ← this file
├── docker-compose.yml
└── Dockerfile
```

---

4. Backend — Spring Boot REST API

4.1 Maven Configuration

File: [modules/cloudsim-web/pom.xml]

The backend is a standard Maven module that inherits from the root `cloudsim-package:7.0.1` POM. It pulls in CloudSim as a local dependency.

Key dependencies:

Group / Artifact	Version	Purpose

`spring-boot-starter-web`	3.3.5	REST controllers, embedded Tomcat
`spring-boot-starter-security`	3.3.5	Filter chain, auth entry points
`spring-boot-starter-data-jpa`	3.3.5	ORM, repositories
`spring-boot-starter-validation`	3.3.5	`@Valid` on DTOs
`jjwt-api/impl/jackson`	0.12.6	JWT creation and validation
`sqlite-jdbc`	3.46.1.3	SQLite JDBC driver
`hibernate-community-dialects`	—	SQLite Hibernate dialect
`bucket4j-core`	8.10.1	Token-bucket rate limiting
`h2` (test scope)	—	In-memory DB for unit tests

The `spring-boot-maven-plugin` repackages the output into an executable fat JAR. The compiler is set to emit `-parameters` so that Spring can resolve method parameter names via reflection (required for controller binding).

All JUnit 5 artifacts are explicitly pinned to matching 5.11.x versions to prevent `NoSuchMethodError` from version mismatches.

4.2 Application Properties

File: [modules/cloudsim-web/src/main/resources/application.properties]

```
server.port=8080

# SQLite — auto-created under the user's home directory
spring.datasource.url=jdbc:sqlite:${user.home}/.cloudsim/cloudsim.db
spring.datasource.driver-class-name=org.sqlite.JDBC
spring.jpa.database-platform=org.hibernate.community.dialect.SQLiteDialect
spring.jpa.hibernate.ddl-auto=update   # creates tables if they don't exist

# JWT — override the secret in production via environment variable
app.jwt.secret=${APP_JWT_SECRET:cloudsim-default-secret-change-in-production-32ch}

# Suppress CloudSim's verbose console output at the API layer
logging.level.org.cloudbus.cloudsim=WARN
```

Notable points:

- The database file is created automatically at `~/.cloudsim/cloudsim.db` on first start.
- `ddl-auto=update` means new columns or tables are added on startup without dropping existing data.
- The JWT secret must be at least 32 characters. Set `APP_JWT_SECRET` as an environment variable in any real deployment.

4.3 Authentication — JWT Cookie Flow

Files: auth/AuthController.java, auth/AuthService.java, auth/JwtUtil.java

The app uses **stateless JWT authentication** stored in an `httpOnly` cookie (not localStorage, which is vulnerable to XSS).

Registration / Login flow:

1. `POST /api/auth/register` — creates a new user, stores `bcrypt(password, strength=12)`.
2. `POST /api/auth/login` — validates password, calls `JwtUtil.generate(userId, email)`.
3. The generated JWT is written as a `Set-Cookie: access_token=...; HttpOnly; SameSite=Strict` response header with an 8-hour TTL.
4. The browser stores the cookie and sends it automatically on every subsequent request.

JWT contents (HS384 algorithm, 8-hour expiry):

```
{
  "sub": "4",              ← userId as string
  "email": "user@test.com",
  "iat": 1777113364,
  "exp": 1777142164
}
```

Logout: `POST /api/auth/logout` clears the cookie by setting `Max-Age=0`.

`GET /api/auth/me`: Returns `{ userId, email }` if the cookie is valid; used by the frontend to check login state.

JwtUtil:

- Secret must be ≥ 32 bytes; enforced at startup.
- `generate(userId, email)` → signed JWT string.
- `isValid(token)` → boolean (catches `JwtException`).
- `getUserId(token)` → `Long` extracted from `sub` claim.
- `getEmail(token)` → `String` from custom `email` claim.

4.4 Security Filter Chain

File: config/SecurityConfig.java

Spring Security is configured as a **stateless** filter chain:

```
Incoming request
    │
    ▼
JwtAuthFilter (OncePerRequestFilter)
    │  extracts access_token cookie
    │  calls JwtUtil.isValid()
    │  if valid → sets UsernamePasswordAuthenticationToken in SecurityContextHolder
    │     principal = userId (Long)
    │     details   = email (String)
    ▼
DispatcherServlet (routes to controller)
    │
```

```
    ▼
Controller method receives @AuthenticationPrincipal Long userId
```

Public endpoints (no JWT needed):

- `POST /api/auth/register`
- `POST /api/auth/login`
- `GET /api/templates`
- `GET /api/templates/**`

All other endpoints require a valid JWT cookie. If no valid cookie is present, Spring Security returns:

```
{ "error": "Authentication required" }
```

with HTTP 401.

CORS: Configured to allow `http://localhost:5173` (Vite dev server) with `credentials: true` so cookies are sent cross-origin during development.

CSRF: Disabled — the API is stateless and cookies are `SameSite=Strict`, which prevents cross-site request forgery on modern browsers.

4.5 Simulation Lifecycle

Files: [simulation/SimulationController.java](), [simulation/SimulationService.java]()

```
POST /api/simulations   (body = SimulationConfigDto)
         |
         ▼
  SimulationService.submit()
         |  1. validates config (SimulationConfigValidator)
         |  2. creates SimulationRun entity, status = QUEUED
         |  3. serialises config to JSON, saves to DB
         |  4. submits Callable to single-thread ExecutorService
         |  5. returns { simulationId, status: "QUEUED" }   ← HTTP 202
         |
         ▼  (in background thread)
  SimulationService.executeRun()
         |  status → RUNNING
         |  calls SimulationRunner.run(configDto)
         |  status → COMPLETED (or FAILED)
         |  serialises RunResult → JSON → stored in DB
         |
Frontend polls:
  GET /api/simulations/{id}   every 1 second
         |  returns SimulationResultDto with current status
         |  frontend stops polling when status = COMPLETED or FAILED
```

Why a single-thread executor? CloudSim uses static Java state — `CloudSim.init()` registers global listeners and a global clock. Two concurrent calls would corrupt each other's state. The single-thread executor ensures at most one simulation runs at a time.

GET /api/simulations returns a summary list for the authenticated user (newest first), including status, cloudlet counts, and makespan.

GET /api/simulations/{id} returns the full `SimulationResultDto`:

- Summary statistics (makespan, cost, throughput, SLA violations, energy, failures)
- Per-cloudlet results (timing, cost, SLA flag, failure reason)
- Per-VM aggregates (cloudlet count, avg exec time, CPU utilisation)
- Per-host energy (if power model was configured)
- Failure events (if failure injection was configured)
- Raw simulation logs (truncated at 5 MB)

4.6 SimulationMapper — Java Object Construction

File: simulation/SimulationMapper.java

This class translates the incoming JSON DTOs into actual CloudSim Java objects. It is the bridge between the REST layer and the simulation engine.

`buildHosts(DatacenterConfigDto)` → `List<Host>`

For each host in the config:

1. Creates `PeProvisionerSimple` instances for each processing element (PE), setting MIPS rating.
2. Creates the VM scheduler based on the `vmScheduler` enum:
   - `TIME_SHARED` → `VmSchedulerTimeShared`
   - `SPACE_SHARED` → `VmSchedulerSpaceShared`
   - `TIME_SHARED_OVERSUBSCRIPTION` → `VmSchedulerTimeSharedOverSubscription`
3. If `powerModel` is set (LINEAR, SQUARE, CUBIC) and `maxPowerW > 0`, creates a `PowerHost` with a `PowerModelLinear`, `PowerModelSquare`, or `PowerModelCubic`. The idle power threshold is `maxPowerW × idlePowerPercent`.
4. Otherwise creates a standard `Host`.

`buildDatacenter(DatacenterConfigDto, List<Host>)` → `Datacenter`

1. Creates `DatacenterCharacteristics` with architecture, OS, VMM, cost parameters (CPU/second, memory/MB, storage/MB, bandwidth/Mbps).
2. Creates VM allocation policy:
   - `SIMPLE` → `VmAllocationPolicySimple` (first-fit)
   - `SIMPLER` → `VmAllocationPolicySimpler`
3. Returns a `Datacenter` with `schedulingInterval`.

`buildVm(id, brokerId, VmConfigDto)` → `Vm`

Creates a VM with MIPS, PE count, RAM, BW, image size, and a cloudlet scheduler:

- `TIME_SHARED` → `CloudletSchedulerTimeShared` (concurrent, round-robin)
- `SPACE_SHARED` → `CloudletSchedulerSpaceShared` (FCFS, one at a time)
- `DYNAMIC_WORKLOAD` → `CloudletSchedulerDynamicWorkload`

`buildCloudlet(id, brokerId, CloudletConfigDto)` → `Cloudlet`

Creates a cloudlet with length (MI), PE count, file/output sizes, and utilization models per resource:

- `FULL` → `UtilizationModelFull` (100% always)

- `NULL` → `UtilizationModelNull` (0% always)
- `STOCHASTIC` → `UtilizationModelStochastic` (random each tick)

If `assignedVmId` is set, the cloudlet is bound to a specific VM via `cloudlet.setVmId()`.

4.7 SimulationRunner — Executing CloudSim

File: [simulation/SimulationRunner.java]

This is the core execution engine. Every simulation call goes through `SimulationRunner.run(configDto)`.

Execution steps:

```
1. Reset static CloudSim entity ID counters
2. Redirect System.out to a StringBuilder (to capture logs)
3. CloudSim.init(1, Calendar.getInstance(), traceFlag)
4. Build datacenters from config → list of Datacenter + HostEntry records
5. Create DatacenterBroker
6. Build VMs from config → submit to broker
7. Build cloudlets from config → submit to broker
8. CloudSim.startSimulation()
9. CloudSim.stopSimulation()
10. Collect completed cloudlets from broker
11. --- Post-processing ---
12.   a. Map cloudlets to CloudletResultDto (timing, cost, SLA check)
13.   b. Failure injection (stochastic, post-sim)
14.   c. Energy extraction from PowerHosts
15.   d. Restore System.out, return RunResult
```

Failure injection (step 13b):

Failure injection is done *post-simulation* as a statistical overlay, not by actually crashing the CloudSim simulation mid-run. This avoids needing to modify CloudSim internals.

For each host that has `failureProbability > 0`:

1. Draw a random number. If `random < probability`, the host "fails".
2. Determine failure time: either the explicit `failureTimeS` from config, or a random value in `[0, makespan]`.
3. Find all cloudlets that were executing on VMs hosted by this host at the failure time (i.e. `startTime ≤ failureTimeS ≤ finishTime`).
4. Mark those cloudlets as `FAILED` with reason `"Host failure at T=Xs"`.
5. Record a `FailureEventDto` (host ID, datacenter name, failure time, affected cloudlet IDs).

Energy extraction (step 14c):

For each `PowerHost`:

1. If `getStateHistory()` is non-empty (CloudSim recorded utilisation snapshots), compute average utilisation from history.
2. Evaluate the power model at that utilisation: `avgPowerW = powerModel.getPower(avgUtilisation)`.
3. Also evaluate at 100% for peak power.

4. Compute energy: `energyWh = avgPowerW × (makespan / 3600)`.
5. If no history (CloudSim didn't record), estimate using MIPS ratio as proxy for utilisation.

4.8 Parameter Sweep

Files: sweep/SweepController.java, sweep/SweepService.java

A parameter sweep runs the same base simulation repeatedly, varying one dimension each time, to see how performance metrics change as that parameter scales.

Endpoint: `POST /api/sweeps` — synchronous; returns when all points are done.

Available dimensions:

Dimension	What changes
`VM_COUNT`	Number of VMs (repeats the first VM config N times)
`VM_MIPS`	MIPS rating of all VMs
`HOST_COUNT`	Number of hosts in each datacenter (repeats first host config N times)
`CLOUDLET_COUNT`	Number of cloudlets (repeats first cloudlet config N times)
`CLOUDLET_LENGTH`	Length (MI) of all cloudlets

How it works:

```
value = start
while (value <= end && iterations < 10) {
    modifiedConfig = applyDimension(baseConfig, dimension, value)
    result = runner.run(modifiedConfig)
    point = SweepPointResult(paramValue=value, summary=buildSummary(result))
    points.add(point)
    value += step
    iterations++
}
```

Each point records the full `SummaryStats` (makespan, avg exec time, throughput, cost, energy, etc.). The frontend plots these as a line chart.

Max 10 points — hardcoded limit to prevent very long-running sweeps from blocking the single-thread executor for too long.

Sweep runs synchronously on the HTTP thread (unlike regular simulations which are async). The frontend shows a loading spinner for the full sweep duration.

4.9 Templates

File: templates/TemplateService.java

Nine pre-built templates mirror the original CloudSim examples included in `modules/cloudsim-examples/`. They are hardcoded in Java (not read from a database) so they are always available without any seed data.

Template ID	Name	Key setup
`example1`	Basic — 1 VM, 1 Cloudlet	1 DC · 1 host (1 PE, 1000 MIPS) · 1 VM · 1 cloudlet
`example2`	2 VMs, bound cloudlets	VM-to-cloudlet binding via `assignedVmId`
`example3`	Heterogeneous VMs	2 hosts with different MIPS (1000 and 2000)
`example4`	Multi-datacenter	2 DCs · SpaceShared VM scheduler
`example5`	Multi-broker	2 DCs · separate broker
`example6`	Scale — 20 VMs, 40 cloudlets	4 quad-core + 2 dual-core hosts
`example7`	Pause/Resume	TimeSharedOverSubscription scheduler
`example8`	GlobalBroker	Runtime broker assignment
`example9`	Scheduler comparison	TimeShared vs SpaceShared cloudlet scheduling
`blank`	Custom (empty)	Minimal single-host single-VM starting point

`GET /api/templates` returns the list with IDs, names, and descriptions. `GET /api/templates/{id}` returns the full `SimulationConfigDto` for that template, ready to prefill the form.

4.10 Database Schema

Engine: SQLite · **Location**: `~/.cloudsim/cloudsim.db` · **DDL**: auto-managed by Hibernate

Table: `users`

Column	Type	Notes
`id`	INTEGER PK	Auto-increment
`email`	TEXT UNIQUE	Login identifier
`password_hash`	TEXT	BCrypt(strength=12)
`created_at`	TEXT	ISO-8601 timestamp

Table: `simulation_runs`

Column	Type	Notes
`id`	INTEGER PK	Auto-increment
`user_id`	INTEGER FK	References `users.id`
`name`	TEXT	Simulation name
`template_id`	TEXT	Template used, or null
`status`	TEXT	`QUEUED` / `RUNNING` / `COMPLETED` / `FAILED`
`config_json`	TEXT	Full `SimulationConfigDto` as JSON

`result_json`	TEXT	Full `StoredResult` as JSON (after completion)
`logs`	TEXT	Captured CloudSim stdout (max 5 MB)
`created_at`	TEXT	ISO-8601
`completed_at`	TEXT	ISO-8601, nullable

Table: `saved_configs`

Column	Type	Notes
`id`	INTEGER PK	Auto-increment
`user_id`	INTEGER FK	Owner
`name`	TEXT	Config name
`config_json`	TEXT	Full `SimulationConfigDto` as JSON
`created_at`	TEXT	ISO-8601

Results are stored as a single JSON blob (`result_json`) rather than normalised rows. This keeps the schema simple — at simulation scale (tens of cloudlets) there is no query performance concern, and the full result can be deserialized in one call.

4.11 REST API Contract

Auth

Method	Path	Body	Response	Auth Required
POST	`/api/auth/register`	`{ email, password }`	`{ userId, email }`	No
POST	`/api/auth/login`	`{ email, password }`	`{ userId, email }` + sets cookie	No
POST	`/api/auth/logout`	—	`{}` + clears cookie	No
GET	`/api/auth/me`	—	`{ userId, email }`	Yes (cookie)

Templates

Method	Path	Response
GET	`/api/templates`	`[{ id, name, description }]`
GET	`/api/templates/{id}`	Full `SimulationConfigDto`

Simulations

Method	Path	Body	Response	HTTP

POST	`/api/simulations`	`SimulationConfigDto`	`{ simulationId, status }`	202
GET	`/api/simulations`	—	`[SimulationSummaryDto]`	200
GET	`/api/simulations/{id}`	—	`SimulationResultDto`	200

Saved Configs

Method	Path	Body	Response
POST	`/api/configs`	`SimulationConfigDto`	`{ configId, name, createdAt }`
GET	`/api/configs`	—	`[{ configId, name, createdAt }]`
GET	`/api/configs/{id}`	—	`SimulationConfigDto`
DELETE	`/api/configs/{id}`	—	204 No Content

Parameter Sweeps

Method	Path	Body	Response
POST	`/api/sweeps`	`SweepConfigDto`	`SweepResultDto`

---

5. Frontend — React TypeScript App

5.1 Tech Stack

Library	Version	Role
React	18.3.1	UI framework
TypeScript	5.6.3	Type safety
Vite	5.4.11	Dev server + build tool
React Router v6	6.28.0	Client-side routing
TanStack Query v5	5.59.20	Server state, caching, polling
react-hook-form	7.53.2	Form state and validation
Zod	3.23.8	Schema validation
Tailwind CSS	3.4.14	Utility-first CSS
shadcn/ui	—	Radix UI + Tailwind component system
Recharts	2.13.3	Data visualisation (bar, pie, line charts)
lucide-react	0.460.0	Icons
Zustand	5.0.1	Lightweight client state store

5.2 Vite Dev Proxy

File: [web/vite.config.ts]

During development, Vite runs on port 5173. Any request to `/api/**` is proxied to `http://localhost:8080`. A `proxyReq` event handler copies request cookies to the proxied request so JWT authentication works seamlessly.

```
proxy: {
  '/api': {
    target: 'http://localhost:8080',
    changeOrigin: true,
    configure: (proxy) => {
      proxy.on('proxyReq', (proxyReq, req) => {
        if (req.headers.cookie) {
          proxyReq.setHeader('Cookie', req.headers.cookie)
        }
      })
    },
  },
}
```

In production, the React build output (`web/dist/`) is served by the Spring Boot server as static files, so no proxy is needed.

5.3 API Client — lib/api.ts

File: [web/src/lib/api.ts]

All HTTP calls go through a single `request<T>()` function:

- Sets `credentials: 'include'` so cookies are sent.
- Sets `Content-Type: application/json` for POST bodies.
- On non-2xx: throws `ApiError` with the status code and parsed message. On 429, extracts `Retry-After`.

Typed API clients:

```
authApi    — register, login, logout, me
simulationApi — submit, list, get
templateApi   — list, get
savedConfigApi — save, list, get, delete
sweepApi      — run
```

Type definitions live in this file covering every DTO the backend returns: `SimulationConfig`, `SimulationResult`, `SummaryStats`, `CloudletResult`, `VmStats`, `HostEnergy`, `FailureEvent`, `SweepResult`, etc.

5.4 React Hooks Layer

Files: [web/src/hooks/useAuth.ts], [web/src/hooks/useSimulations.ts]

All data fetching is done through custom hooks that wrap TanStack Query. Components never call `fetch` directly.

Auth hooks:

Hook	What it does
`useAuth()`	`useQuery` → `GET /api/auth/me`; staleTime 5 min; returns null on error
`useLogin()`	`useMutation` → `POST /api/auth/login`; on success: updates query cache
`useRegister()`	`useMutation` → `POST /api/auth/register`; on success: invalidates `auth` query
`useLogout()`	`useMutation` → `POST /api/auth/logout`; on success: clears all query cache

Simulation hooks:

Hook	What it does
`useSimulations()`	`useQuery` → list of user's simulations
`useSimulationResult(id)`	`useQuery` → single result; **auto-polls every 1 second** while status is QUEUED or RUNNING; stops when COMPLETED/FAILED
`useSubmitSimulation()`	`useMutation` → submit config; on success: invalidates simulations list
`useTemplates()`	`useQuery` → template list; `staleTime: Infinity` (templates never change)
`useTemplate(id)`	`useQuery` → single template config; only enabled when id is non-null
`useSavedConfigs()`	`useQuery` → user's saved configs
`useSavedConfig(id)`	`useQuery` → single saved config
`useSaveConfig()`	`useMutation` → save config; invalidates saved configs list
`useDeleteSavedConfig()`	`useMutation` → delete; invalidates saved configs list

The polling strategy in `useSimulationResult`:

```
refetchInterval: (query) => {
  const status = query.state.data?.status
  if (status === 'COMPLETED' || status === 'FAILED') return false
  return 1000
}
```

5.5 Page: Landing

File: [web/src/pages/Landing.tsx]

The public homepage. Describes what CloudSim Web is, lists features, and links to `/login` and `/register`. No authentication required.

5.6 Page: Login / Register

Files: [web/src/pages/Login.tsx], [web/src/pages/Register.tsx]

Simple forms using `react-hook-form`. On success, both redirect to `/dashboard`. Show inline error messages from the API (e.g. "Email already registered", "Invalid credentials").

5.7 Page: Dashboard

File: [web/src/pages/Dashboard.tsx]

The main landing page after login. Shows:

- A list of all the user's simulations (name, date, status badge, cloudlet count, makespan).
- Each row is a link to the simulation result page.
- Skeleton loaders while `useSimulations()` is loading.
- Empty state card with a "New simulation" CTA.
- Header navigation to all other sections: Analytics, Compare, Sweep, Saved Configs.
- Dark mode toggle.

5.8 Page: New Simulation

File: [web/src/pages/NewSimulation.tsx]

The main configuration form. Built with `react-hook-form` using nested `useFieldArray` for datacenters → hosts, VMs, and cloudlets.

Sections:

1. **Simulation Name** — required text field.
2. **Load Template** — buttons for each template; clicking one calls `useTemplate(id)` and resets the form with the loaded config.
3. **Datacenters** — collapsible sections (one per DC). Each DC has:
   - Name, architecture, OS, VMM, time zone, scheduling interval
   - Cost parameters: `costPerSec`, `costPerMem`, `costPerStorage`, `costPerBw`
   - VM allocation policy (Simple / Simpler)
   - **Tier 3 Network** section (dashed blue border): network latency and bandwidth fields — stored and displayed but not simulated by CloudSim core
   - **Hosts** sub-list with Add/Remove; each host has:
     - Core: MIPS, PEs, RAM, BW, storage, VM scheduler
     - **Tier 2 toggle** (dashed amber border): power model, max/idle power, failure probability, failure time
4. **Virtual Machines** — grid of VM rows; each has MIPS, PEs, RAM, BW, image size, cloudlet scheduler. Bulk generate N VMs from the first default.
5. **Cloudlets** — grid of cloudlet rows; each has length (MI), PEs, file/output size, CPU utilisation model, deadline (for SLA), VM assignment. Bulk generate N cloudlets.
6. **Global Settings** — simulation clock, trace flag.
7. **Actions** — Cancel, Save Config, Run Simulation.

When submitted, `useSubmitSimulation()` fires, gets back `{ simulationId }`, and navigates to `/simulations/{id}`.

Saving a config calls `useSaveConfig()` and shows a toast notification.

Loading a saved config: If navigated from Saved Configs page with `location.state.savedConfigId`, the form loads that config automatically.

5.9 Page: Simulation Result

File: [web/src/pages/SimulationResult.tsx]

Displays all results for a completed simulation. Uses `useSimulationResult(id)` which polls until the simulation finishes.

States rendered:

- Loading skeleton (while initial query loads)
- Spinner + "Simulation running..." (while status is QUEUED or RUNNING)
- Error card (if status is FAILED)
- Full result layout (when COMPLETED)

Summary row 1 (always shown):

- Completed cloudlets / total cloudlets
- Makespan + throughput
- Total cost
- Avg execution time (with p95 and max)

Summary row 2:

- Avg wait time + max
- SLA violations (highlighted amber if > 0)
- Failed cloudlets (highlighted red if > 0)
- Total energy / avg power (if hosts have power model) or min exec time

Tab navigation:

Tab	Content
Results & Charts	Cloudlet table · Per-VM breakdown · Exec time bar chart · Status pie chart · Wait time bar chart · Cost breakdown (CPU vs BW) · Static Gantt chart
Energy / Power	Host energy table · Energy bar chart · (Optional) network config info
Failures	Failure event cards · Failed cloudlet table
Timeline	Animated SVG Gantt with play/pause/scrub
Logs	Raw CloudSim stdout (scrollable, truncated at 5 MB)

Animated Timeline: An SVG Gantt that replays the simulation in real time. Uses `requestAnimationFrame` for smooth animation. Bars change colour as the virtual clock advances:

- Grey = not yet started
- Indigo = currently running
- Green = completed
- Amber = SLA violation

- Red = injected failure

Export: "Download CSV" and "Download JSON" buttons on the Results tab generate client-side blobs without any server call.

5.10 Page: Analytics

File: web/src/pages/Analytics.tsx

Aggregated view across all of a user's completed simulations. Shows trends and distributions:

- Total simulations run, total cloudlets, average makespan
- Makespan trend over time (line chart)
- Throughput distribution
- Cost breakdown across simulations

5.11 Page: Compare

File: web/src/pages/Compare.tsx

Side-by-side comparison of two completed simulations. The user picks two simulations from dropdowns.

Scorecard — a summary of which simulation "wins" in key categories:

- Lower makespan
- Higher throughput
- Lower total cost
- Lower avg wait time
- Lower energy use (only shown if at least one simulation has energy data)

Each win is highlighted; the simulation winning more categories is declared the winner.

Metrics Breakdown table — full numeric comparison with colour-coded better/worse highlighting:

- Makespan, completed/total, avg exec time, p95 exec time
- Avg wait time, max wait time
- Total cost
- Throughput
- SLA violation rate
- Total energy (Wh) — only if energy data present
- Avg power (W) — only if energy data present
- Injected failures — only if failures present

5.12 Page: Parameter Sweep

File: web/src/pages/ParameterSweep.tsx

Lets the user vary one simulation parameter across a range and see how metrics respond.

Configuration form:

- **Base simulation** — a dropdown of completed simulations; loading one fetches its full config to use as the base.
- **Sweep name** — optional label for the sweep.
- **Dimension** — which parameter to vary: VM Count, VM MIPS, Host Count, Cloudlet Count, Cloudlet Length.

- **Start / End / Step** — numeric range. A live preview shows how many points will be generated. If > 10, the form shows an error and disables submission.

Running calls `sweepApi.run()` via `useMutation`. The backend runs all points synchronously and returns the full result. The frontend shows a spinner during the request.

Results:

Chart — a `LineChart` (Recharts) with the parameter value on the X-axis and a selectable metric on the Y-axis. Metric toggle buttons: Makespan, Avg Exec Time, Total Cost, Throughput, Avg Wait Time, Energy.

Raw Data table — one row per sweep point showing all key metrics: makespan, completed/total, avg exec, throughput, cost, energy.

5.13 Page: Saved Configs

File: [web/src/pages/SavedConfigs.tsx]

Lists all configs the user has saved. Each row shows the name and save date with two actions:

- **Load** — navigates to New Simulation with `location.state.savedConfigId` set.
- **Delete** — calls `useDeleteSavedConfig()` with a confirmation step.

---

6. CloudSim Core Concepts

6.1 Domain Objects

Object	Role
`Datacenter`	A physical computing facility. Contains hosts, cost model, scheduling interval.
`DatacenterCharacteristics`	Metadata: architecture, OS, VMM, cost per resource type.
`Host`	A physical server inside a datacenter. Has RAM, BW, storage, PE list, VM scheduler.
`PowerHost`	Extends Host with energy tracking via a power model.
`Pe`	A single processing element (CPU core) with a MIPS rating.
`Vm`	A virtual machine running on a host. Has MIPS, PEs, RAM, BW, cloudlet scheduler.
`Cloudlet`	A unit of work (task) to execute on a VM. Has length in MI, resource requirements.
`DatacenterBroker`	Routes VMs and cloudlets between the user and datacenters.

6.2 Scheduling Policies

VM Allocation Policy (on Datacenter) — decides which host a VM is placed on:

- `VmAllocationPolicySimple` — first-fit: place VM on the first host with enough resources.
- `VmAllocationPolicySimpler` — simplified variant.

VM Scheduler (on Host) — decides how host PEs are shared among VMs:

- `VmSchedulerTimeShared` — VMs share PE time; no over-subscription.
- `VmSchedulerSpaceShared` — each VM gets dedicated PEs; no sharing.
- `VmSchedulerTimeSharedOverSubscription` — time-shared but allows more VMs than PEs.

Cloudlet Scheduler (on VM) — decides how a VM's CPU is shared among cloudlets:

- `CloudletSchedulerTimeShared` — concurrent execution, round-robin.
- `CloudletSchedulerSpaceShared` — FCFS; one cloudlet at a time.
- `CloudletSchedulerDynamicWorkload` — advanced scheduling with dynamic CPU requirements.

6.3 Utilization Models

Applied per-resource (CPU, RAM, bandwidth) on each cloudlet:

- `UtilizationModelFull` — always demands 100% of the resource.
- `UtilizationModelNull` — demands 0% (resource not used).
- `UtilizationModelStochastic` — random demand each simulation tick.

6.4 Power Models

Applied on `PowerHost` instances. Given a CPU utilisation `u ∈ [0, 1]`:

Model	Formula
LINEAR	`P = P_idle + (P_max − P_idle) × u`
SQUARE	`P = P_idle + (P_max − P_idle) × u²`
CUBIC	`P = P_idle + (P_max − P_idle) × u³`

Where `P_idle = P_max × idlePowerPercent`.

---

7. Configuration Parameter Reference

Per Datacenter

Parameter	Type	Default	Description
`name`	string	`Datacenter_0`	Unique datacenter name
`architecture`	string	`x86`	CPU architecture label
`os`	string	`Linux`	OS label
`vmm`	string	`Xen`	Hypervisor label
`timeZone`	double	`10.0`	UTC offset (−12 to +13)
`costPerSec`	double	`3.0`	Cost per CPU-second
`costPerMem`	double	`0.05`	Cost per MB RAM
`costPerStorage`	double	`0.001`	Cost per MB storage
`costPerBw`	double	`0.0`	Cost per Mbps bandwidth

`schedulingInterval`	double	`0.0`	Simulation event interval
`vmAllocationPolicy`	enum	`SIMPLE`	`SIMPLE` or `SIMPLER`
`networkLatencyMs`	double?	null	Tier 3; stored, not simulated
`networkBandwidthMbps`	double?	null	Tier 3; stored, not simulated

Per Host

Parameter	Type	Default	Description
`mips`	double	`1000`	MIPS per PE
`numberOfPes`	int	`1`	CPU core count
`ram`	int	`2048`	RAM in MB
`bw`	long	`10000`	Bandwidth in Mbps
`storage`	long	`1000000`	Storage in MB
`vmScheduler`	enum	`TIME_SHARED`	VM scheduling policy
`powerModel`	enum?	null	Tier 2: `LINEAR`, `SQUARE`, `CUBIC`, or null
`maxPowerW`	double?	null	Tier 2: peak power in watts
`idlePowerPercent`	double?	null	Tier 2: idle power as fraction of max (e.g. 0.4)
`failureProbability`	double?	null	Tier 2: 0–1 chance of host failure
`failureTimeS`	double?	null	Tier 2: when failure occurs; null = random

Per VM

Parameter	Type	Default	Description
`mips`	double	`1000`	Processing speed
`numberOfPes`	int	`1`	Virtual CPU count
`ram`	int	`512`	RAM in MB
`bw`	long	`1000`	Bandwidth in Mbps
`size`	long	`10000`	VM image size in MB
`vmm`	string	`Xen`	Hypervisor label
`cloudletScheduler`	enum	`TIME_SHARED`	Cloudlet scheduling policy

Per Cloudlet

Parameter	Type	Default	Description
`length`	long	`400000`	Work in million instructions

`numberOfPes`	int	`1`	CPU cores required
`fileSize`	long	`300`	Input data size (bytes)
`outputSize`	long	`300`	Output data size (bytes)
`utilizationModelCpu`	enum	`FULL`	CPU demand model
`utilizationModelRam`	enum	`FULL`	RAM demand model
`utilizationModelBw`	enum	`FULL`	BW demand model
`assignedVmId`	int?	null	Pin to a specific VM; null = auto
`deadline`	double?	null	SLA deadline in seconds; null = no SLA

Global

Parameter	Type	Default	Description
`simulationClock`	double	`11.0`	Max simulation time
`traceFlag`	boolean	`false`	Enable verbose CloudSim event logging

---

8. Advanced Features — Tier 2

8.1 Power Modeling and Energy Tracking

Enable by setting `powerModel` to `LINEAR`, `SQUARE`, or `CUBIC` on a host, then providing `maxPowerW` and `idlePowerPercent`.

Example (a mid-range server):

- `powerModel: LINEAR`
- `maxPowerW: 250`
- `idlePowerPercent: 0.4` → idle power = 100 W

At average 60% utilisation: `P = 100 + (250 − 100) × 0.6 = 190 W`

After a simulation with 400-second makespan: `energy = 190 × (400/3600) = 21.1 Wh`

The **Energy / Power tab** on the result page shows per-host energy, average power, peak power, and utilisation bars.

The **Compare page** adds an energy row to the scorecard and breakdown table when any simulation has energy data.

The **Parameter Sweep** includes `totalEnergyWh` as a selectable metric in the line chart.

8.2 Failure Injection

Enable by setting `failureProbability > 0` on a host (Tier 2 section of the host editor).

- `failureProbability: 1.0` + `failureTimeS: 400` → guaranteed host crash at T=400s.
- `failureProbability: 0.5` + `failureTimeS: null` → 50% chance of crash at a random time.

Cloudlets running on that host at the failure time are marked FAILED with a reason string. The **Failures tab** shows which host failed, when, and which cloudlets were affected. The cloudlet table highlights failed rows in red with a flame icon.

8.3 SLA Deadlines

Set `deadline` in seconds on any cloudlet. If the cloudlet finishes after its deadline, it is flagged as `slaViolated = true`.

The result page shows SLA violations as:

- A count in the summary cards (highlighted amber if > 0).
- A warning triangle icon in the cloudlet table SLA column.
- Amber-highlighted bars in the Wait Time chart.

The Compare page shows `slaViolationRate` (violations / total) as a percentage in the breakdown table.

---

9. Key Design Decisions

Single-thread executor for CloudSim

CloudSim's static state (`CloudSim.init()` , global event queues) cannot be shared across threads. Rather than spawning sub-processes (which would be heavier and harder to integrate), all simulation runs are submitted to a `newSingleThreadExecutor`. Only one simulation runs at a time; others queue. For a research/demo tool this is acceptable.

SQLite with JPA

SQLite was chosen for zero-infrastructure local development. The JPA layer means switching to Postgres is a one-line change in `application.properties`. The Hibernate `update` DDL mode creates tables on first start without any migration scripts.

Post-simulation failure injection

CloudSim does not have a native "host crash" API. Rather than patching the core (which we must not touch), failure injection is applied as a statistical overlay after the simulation finishes: we look at which cloudlets were executing on the host at the failure time and retroactively mark them as FAILED. This is an approximation but sufficient for demo and educational purposes.

Synchronous sweep vs async simulation

Regular simulations are async (202 Accepted + poll) because they might take seconds. Parameter sweeps are synchronous because: (a) they are bounded to 10 points, (b) the sweep itself needs the executor, and (c) returning a streaming result would add significant complexity for marginal benefit.

httpOnly cookie over localStorage

Storing the JWT in an `httpOnly` cookie prevents JavaScript from reading it, protecting against XSS attacks. `SameSite=Strict` also prevents the cookie from being sent in cross-site requests.

Result JSON blob vs normalised rows

The full simulation result (cloudlets, energy, failures) is stored as a single JSON blob in `result_json`. At simulation scale (tens of cloudlets) normalised rows would add schema complexity without any query

benefit. If the project were to scale to thousands of cloudlets per run, normalisation would become worthwhile.

---

10. How to Run the Project

Prerequisites

- Java 21+
- Maven 4.0.0+
- Node.js 18+ and npm

Start the backend

```
# From the project root
mvn package -pl modules/cloudsim-web -am -DskipTests

java -jar modules/cloudsim-web/target/cloudsim-web-7.0.1.jar
# Starts on http://localhost:8080
# Database auto-created at ~/.cloudsim/cloudsim.db
```

To override the JWT secret (recommended for any real use):

```
java -DAPP_JWT_SECRET=your-32-character-secret-here \
     -jar modules/cloudsim-web/target/cloudsim-web-7.0.1.jar
```

Start the frontend

```
cd web
npm install
npm run dev
# Starts on http://localhost:5173
```

Open `http://localhost:5173` in a browser. Register a new account and start creating simulations.

Production build

```
# Build frontend
cd web && npm run build    # outputs to web/dist/

# The Spring Boot JAR already serves static files from classpath:/static/
# Copy web/dist/ into modules/cloudsim-web/src/main/resources/static/ before
packaging
# then: java -jar modules/cloudsim-web/target/cloudsim-web-7.0.1.jar
# serves both API and frontend on port 8080
```

Docker

```
docker-compose up --build
```

---

11. Pre-built Templates Reference

These reproduce the original CloudSim example programs. Load any of them from the "Load Template" section on the New Simulation page.

Example 1 — Simplest possible simulation

- 1 datacenter · 1 host (1000 MIPS, 1 PE, 2 GB RAM)
- 1 VM (1000 MIPS, 1 PE, 512 MB)
- 1 cloudlet (400,000 MI)
- Use for: sanity check, baseline, first experiment.

Example 2 — VM-to-cloudlet binding

- 1 datacenter · 1 host · 2 VMs · 2 cloudlets
- Each cloudlet explicitly assigned to one VM via `assignedVmId`.
- Use for: demonstrating explicit placement vs auto-assignment.

Example 3 — Heterogeneous hosts

- 1 datacenter · 2 hosts with different MIPS (1000 and 2000)
- 2 VMs with different MIPS · 2 cloudlets
- Use for: demonstrating how faster hosts complete work earlier.

Example 4 — Multi-datacenter, SpaceShared scheduling

- 2 datacenters · SpaceShared VM scheduler on hosts
- Use for: multi-datacenter routing, comparing SpaceShared vs TimeShared.

Example 5 — Two datacenters, single broker

- 2 datacenters, VMs distributed across them
- Use for: broker routing in a multi-DC environment.

Example 6 — Scale: 20 VMs, 40 cloudlets

- 2 datacenters · 4 quad-core hosts + 2 dual-core hosts
- 20 VMs · 40 cloudlets
- Use for: performance under load, Gantt chart with many cloudlets.

Example 7 — TimeShared over-subscription

- Quad-core + dual-core hosts · `TIME_SHARED_OVERSUBSCRIPTION` scheduler
- Use for: demonstrating resource contention and overcommit.

Example 8 — Global broker

- 2 datacenters · 10 VMs · 20 cloudlets
- Use for: large-scale broker coordination.

Example 9 — Scheduler comparison

- 1 datacenter · 2 VMs with different cloudlet schedulers (TimeShared vs SpaceShared)
- 6 cloudlets distributed between them
- Use for: side-by-side scheduling policy comparison.

12. Data Flow — End to End

```
User fills in New Simulation form
        │
        ▼
react-hook-form → validates fields (required, numeric types)
        │
        ▼
onSubmit() → useSubmitSimulation().mutateAsync(config)
        │
        ▼  POST /api/simulations
Backend: JwtAuthFilter validates cookie → userId extracted
        │
        ▼
SimulationController.submit(@AuthenticationPrincipal userId, config)
        │
        ▼
SimulationService.submit():
  • SimulationConfigValidator.validate(config)
  • SimulationRun entity created, status=QUEUED, saved to DB
  • ExecutorService.submit(→ executeRun())
  • Return { simulationId, status: "QUEUED" }
        │
        ▼  HTTP 202 → frontend navigates to /simulations/{id}
        │
Frontend: useSimulationResult(id) starts polling GET /api/simulations/{id} every 1s
        │
        ▼  (background thread in ExecutorService)
SimulationService.executeRun():
  • status → RUNNING (saved)
  • SimulationRunner.run(config):
      CloudSim.init()
      Build Datacenters → Hosts → VMs → Cloudlets
      CloudSim.startSimulation()
      Collect results
      Inject failures (stochastic)
      Extract energy (PowerHosts)
      Capture logs
  • status → COMPLETED
  • result_json serialised and saved to DB
        │
        ▼  next poll returns status=COMPLETED with full SimulationResultDto
Frontend: polling stops → renders result page
  • Summary cards (makespan, cost, SLA, energy)
  • Tab: Results — cloudlet table + charts + Gantt
  • Tab: Energy — host power table + bar chart
  • Tab: Failures — failure events + failed cloudlets
  • Tab: Timeline — animated SVG Gantt
  • Tab: Logs — raw CloudSim stdout
```